BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 3

Roll no:51 PRN No:12412161 Name of Student:Utkarsh Salunkhe

Subject Name : Deep Learning

Problem Statement : Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm :

1.Start
2.Import the required libraries

- Import TensorFlow/Keras, Pandas, Matplotlib, and the required Scikit-learn modules.

3.Load the Wine dataset

- Read the Wine dataset.
- Separate the input features (X) and target labels (y).

4.Preprocess the data

- Split the dataset into training and testing sets.
- Standardize (normalize) the feature values using StandardScaler.

5.Create the Neural Network model

- Add an input layer with 13 input features.
- Add hidden layers using the ReLU activation function.

- Add an output layer with 3 neurons using the Softmax activation function.

6.Compile the model

- Use the Adam optimizer with the desired learning rate.
- Use Sparse Categorical Crossentropy as the loss function.
- Use Accuracy as the evaluation metric.

7.Train the model

- Perform forward propagation to compute predictions.
- Perform backpropagation to calculate gradients and update weights.
- Repeat the process for the specified number of epochs.

8.Evaluate the model

- Test the trained model on the testing dataset.
- Record the test accuracy and loss.

9.Analyze the effect of learning rate

- Train the model using different learning rates (e.g., 0.1, 0.01, 0.001).
- Compare the accuracy obtained for each learning rate.

10.Analyze the effect of epochs

- Train the model using different numbers of epochs (e.g., 20, 50, 100).
- Compare the accuracy obtained for each epoch value.

11.Visualize the results

- Plot the Training Accuracy vs. Validation Accuracy graph.
- Plot the Training Loss vs. Validation Loss graph.
- Plot comparison graphs for different learning rates and epochs.

12.Display the final model performance

- Print the final test accuracy and loss.

13.Stop

Code :

```
[27] ✓ 0s
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
[28] ✓ 0s
wine = load_wine()

X = wine.data
y = wine.target

print("Features Shape :", X.shape)
print("Labels Shape :", y.shape)
```

```
Features Shape : (178, 13)
Labels Shape : (178,)
```

```
[29] ✓ 0s
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
[30] ✓ 0s
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
[31] ✓ 0s
def create_model(lr):
    model = Sequential([
        Dense(128, activation='relu', input_shape=(13,)),
        Dense(64, activation='relu'),
        Dense(32, activation='relu'),
        Dense(3, activation='softmax')
    ])
    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
```

```
[32] ✓ 36s
learning_rates = [0.1, 0.01, 0.001]
lr_results = {}
for lr in learning_rates:
    print("\nTraining with Learning Rate =", lr)
    model = create_model(lr)
    history = model.fit(
        X_train,
        y_train,

        validation_data=(X_test, y_test),
        epochs=50,
        verbose=0
    )
    loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
    lr_results[lr] = accuracy
    print("Test Accuracy =", accuracy)
```

```
Training with Learning Rate = 0.1
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Test Accuracy = 1.0

Training with Learning Rate = 0.01
Test Accuracy = 1.0

Training with Learning Rate = 0.001
Test Accuracy = 1.0
```

```
[33] ✓ 0s
plt.figure(figsize=(7,5))

plt.bar(
    [str(i) for i in lr_results.keys()],
    lr_results.values()
)
```

```
epochs_list = [5,10,20]

epoch_results = {}

for ep in epochs_list:
    print("\nTraining for", ep, "epochs")
    model = create_model(0.001)

    history = model.fit(
        X_train,
        y_train,
        validation_data=(X_test, y_test),
        epochs=ep,
        verbose=0
    )

    loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
    epoch_results[ep] = accuracy
    print("Accuracy =", accuracy)
```

```
Training for 5 epochs
Accuracy = 0.9444444179534912

Training for 10 epochs
Accuracy = 1.0

Training for 20 epochs
Accuracy = 1.0
```

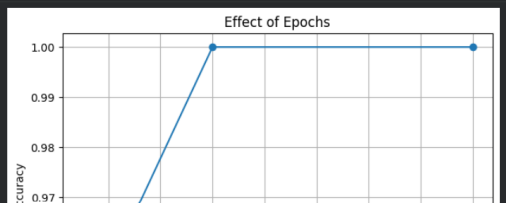
```
plt.figure(figsize=(7,5))

plt.plot(
    list(epoch_results.keys()),
    list(epoch_results.values()),
    marker='o'
)

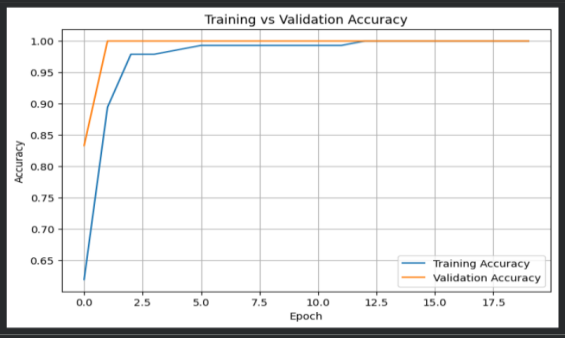
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Effect of Epochs")

plt.grid()

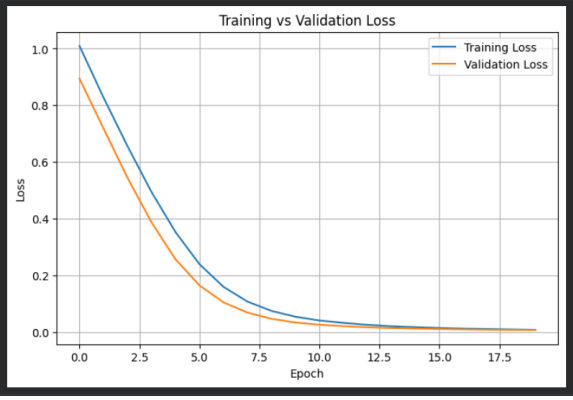
plt.show()
```



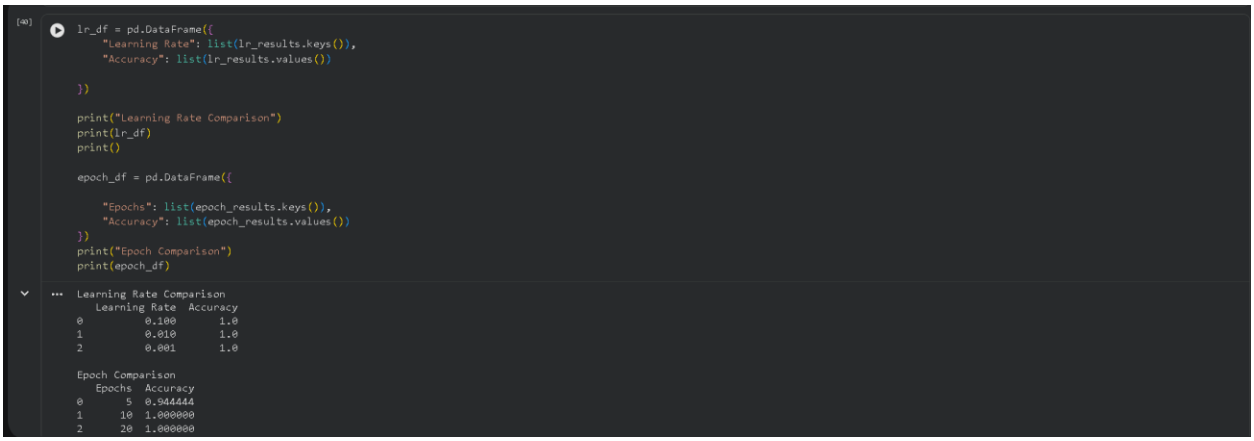

```
plt.show()
```




```
plt.show()
```



Output :



```
lr_df = pd.DataFrame({
    "Learning Rate": list(lr_results.keys()),
    "Accuracy": list(lr_results.values())

})

print("Learning Rate Comparison")
print(lr_df)
print()

epoch_df = pd.DataFrame({

    "Epochs": list(epoch_results.keys()),
    "Accuracy": list(epoch_results.values())
})
print("Epoch Comparison")
print(epoch_df)
```

```
Learning Rate Comparison
   Learning Rate  Accuracy
0          0.100       1.0
1          0.010       1.0
2          0.001       1.0

Epoch Comparison
   Epochs  Accuracy
0       5  0.944444
1      10  1.000000
2      20  1.000000
```